Lesson 14 Teacher's Key: Raising the Stakes

Unreal Engine (optional advanced track) - Instructor Reference Only

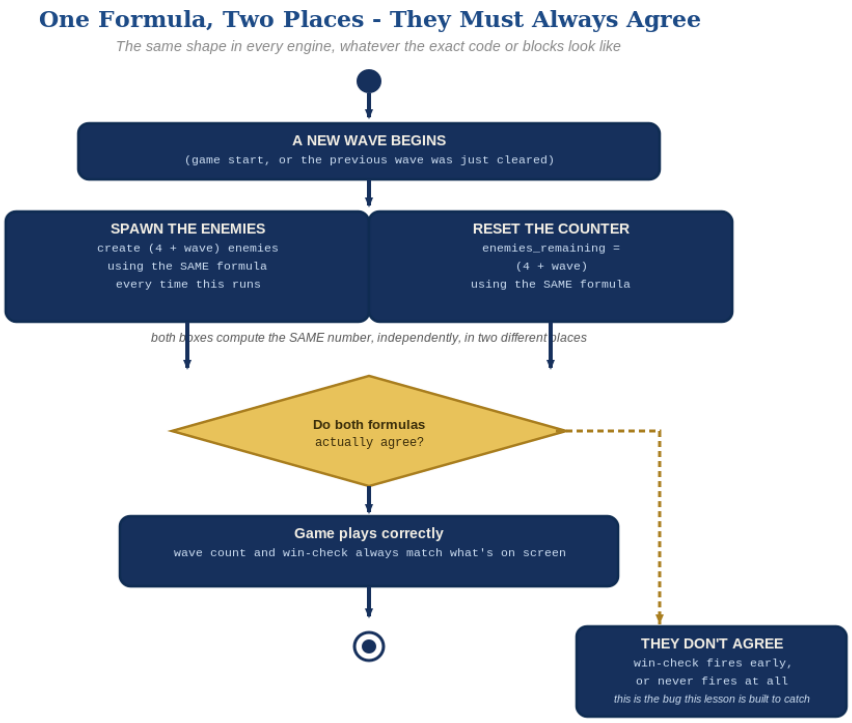
VIDEO GAME CREATION

What This Lesson Teaches

Same idea as every engine this week - replace hardcoded literals with wave-dependent formulas - expressed as Blueprint math nodes inside the SpawnWave Custom Event introduced in Lesson 11.

Level 1 - Full Worked Solution

- (4 + Wave) as the For Loop's end value and as EnemiesRemaining's reset value must be identical - both live inside the same SpawnWave graph, making it easy to verify they agree.
- No new variables are needed for Level 1 - this is purely a formula change inside a Custom Event that already exists.
- Wave 1 still spawns exactly 5 (4 + 1 = 5), so nothing about the game's opening feel changes - only later waves escalate.



Whether it's two places (most engines) or the loop and the reset living in genuinely different scripts (Godot, GameMaker), the rule is the same: one formula, referenced everywhere.

VIDEO GAME CREATION

Whether the spawn loop and the reset live in two places or two genuinely different scripts, the rule is the same: one formula, referenced everywhere.

Why This Structure

- Computing (4 + Wave) once via a shared math node chain (rather than two separate calculations) keeps the For Loop and EnemiesRemaining tied to the exact same value - the same discipline as every other engine this week, expressed as Blueprint wiring instead of a stored variable.
- Setting EnemySpeed on each spawned BP_Enemy right after Spawn Actor from Class (rather than changing BP_Enemy's own default) keeps the scaling logic entirely inside SpawnWave - BP_Enemy's Event Tick movement stays exactly as simple as Lesson 7 left it.

Level 2 / Level 3 Talking Points

- Ask students to trace SpawnWave for wave 3 by hand: (4 + 3) = 7 - a good comprehension check for reading Blueprint math wiring.
- For students on the C++ track: point out that the identical (4 + Wave) formula, expressed as a plain arithmetic expression instead of Blueprint nodes, is conceptually the exact same idea - worth naming that parallel explicitly.

Common Student Mistakes

- Using (4 + Wave) as the For Loop's end value without the "- 1" adjustment For Loop nodes typically need (since For Loop's end value is inclusive) - resulting in one extra enemy per wave. Double-check against your specific For Loop node's exact semantics.
- Updating the For Loop's count but forgetting to update EnemiesRemaining (or vice versa).
- Setting EnemySpeed on BP_Enemy's class defaults instead of the just-spawned instance, which would affect ALL enemies at once rather than scaling per wave.